

From Static Embeddings to Pretrained Transformers: Smiley Prediction in Twitter Sentiment Data

Xinxian Ma, Jinghao Zheng, Rongxiao Qu
Department of Computer Science, EPFL, Switzerland

I. INTRODUCTION

Social media platforms such as Twitter have become a major channel for users to express their opinions. In this paper, the task is to predict whether a tweet originally contained a positive “:)” or a negative “:(” smiley, using only the remaining text.

This is challenging, as the model must rely purely on short, noisy, and informal text with slang, abbreviations, and inconsistent spelling.

We treat this problem as a binary text classification task and compare lightweight neural baselines with modern pretrained language models. First, we build two compact models based on GloVe and FastText embeddings, providing fast and interpretable reference points. Then, we explore transformer-based models pretrained on large corpora, which are adapted to our dataset by fine-tuning the backbone.

Our experiments highlight the trade-offs between model complexity, training cost, and predictive performance. While transformer-based models achieve higher accuracy and better robustness to informal language, the lightweight embedding-based models remain attractive due to their simplicity and low computational requirements.

II. GLOVE-BASED MODEL

A. Provided baseline

The provided baseline model is built on static GloVe word embeddings¹. Each tweet is first tokenised into words, and we count how often pairs of words co-occur within the same tweet to construct a co-occurrence matrix $\mathbf{X} \in \mathbf{R}^{|V| \times |V|}$, where $|V|$ is the vocabulary size and $\mathbf{X}_{ij}$ denotes how frequently word i appears together with word j . GloVe then learns two embedding matrices $\mathbf{W}, \mathbf{Z} \in \mathbf{R}^{|V| \times k}$ by approximately factorising $\mathbf{X}$ such that

$$\log \mathbf{X}_{ij} \approx \mathbf{w}_i^\top \mathbf{z}_j + b_i + \tilde{b}_j,$$

where $\mathbf{w}_i$ and $\mathbf{z}_j$ are the k -dimensional embeddings of words w_i and w_j , and $b_i, \tilde{b}_j$ are scalar bias terms. These parameters are learned by gradient-based optimisation, with the baseline embedding dimension set to $k = 20$.

Given these word embeddings, a tweet representation is obtained by averaging the embeddings of all its words:

$$\mathbf{x}_{\text{tweet}} = \tilde{\mathbf{x}}_{\text{tweet}} = \frac{1}{N} \sum_{i=1}^N \mathbf{v}(w_i) = \frac{1}{N} \sum_{i=1}^N \mathbf{w}_i,$$

where $w_i, i = 1, \dots, N$, are the words in the tweet and $\mathbf{v}(w_i) \in \mathbf{R}^{20}$ denotes their embedding vector.

This vector is then fed into a logistic regression classifier. On the validation split, this baseline achieves an accuracy of **60.7%**, and on the AICrowd test set it reaches **59.6%**. This indicates that the model captures some sentiment signal, but there is still substantial room for further improvements.

B. Improvements over the baseline

We introduce several modifications to improve performance while keeping the model relatively lightweight. These changes fall into three main categories, and their individual contributions are empirically validated in Figure 1.

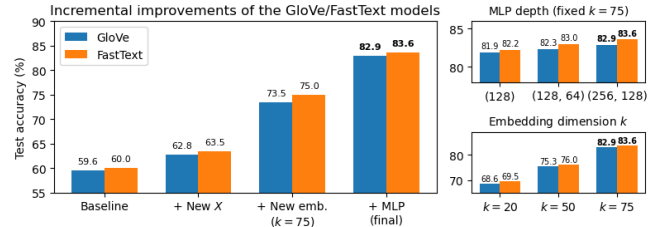


Figure 1. Improvements of the GloVe/FastText model.

1) Input preprocessing:

- **data cleaning:** we remove frequent, sentiment-neutral tokens (e.g. <user> and URLs), which reduces noise in the tweet representation. The effect of this preprocessing is evaluated in Table I.

2) Refining the co-occurrence matrix $\mathbf{X}$:

- **local context window:** we restrict the context to a symmetric window of size 5 around each target word. This focuses the signal on local syntactic and semantic relations and reduces noisy associations between words that only appear together in long tweets.
- **distance-based weighting:** we replace uniform co-occurrence counts by distance-weighted counts: a context word’s contribution is scaled by a decreasing function of its distance from the target. In this way, closer

¹J. Pennington, R. Socher, and C. D. Manning, *GloVe: Global Vectors for Word Representation*, EMNLP 2014.

words receive higher weight, reflecting the intuition that nearby words provide stronger contextual information.

3) Word and sentence embeddings:

- **word embeddings:** We increase the embedding dimension to $k = 75$, in order to capture richer semantic information, and follow the recommendation from the original GloVe work and define the final embedding of word w_i as the sum of its “word” and “context” vectors,

$$\mathbf{v}(w_i) = \mathbf{w}_i + \mathbf{z}_i.$$

This combines information from both target and context roles and yields more informative word representations.

- **sentence embeddings:** we replace pure mean pooling by a concatenation of mean and max pooling over the sequence of word embeddings:

$$\hat{\mathbf{x}}_{\text{tweet},j} = \max_{1 \leq i \leq N} \mathbf{v}_j(w_i) \quad \text{for } j = 1, \dots, k,$$

$$\mathbf{x}_{\text{tweet}} = [\hat{\mathbf{x}}_{\text{tweet}}; \tilde{\mathbf{x}}_{\text{tweet}}] \in R^{150}.$$

Mean pooling captures the overall semantic content of the tweet, while max pooling emphasizes the most salient dimensions across all tokens. This combined pooling scheme enriches the tweet representation.

4) Classifier:

- **non-linearity:** we replace the original linear logistic regression classifier with a MLP in order to capture nonlinear interactions between embedding dimensions. This is particularly important after increasing the embedding size, as a higher-dimensional representation can encode richer patterns.

C. Experiment

This leads to an accuracy of **82.9%** on the test set. As shown in Figure 1, the largest gains come from the new embedding scheme and the MLP classifier. We interpret this as an improvement in the representational capacity of the model: increasing the embedding dimension, introducing non-linear decision boundaries, and combining mean and max pooling all allow the network to encode richer information. We also experimented with even larger embedding sizes and deeper MLPs, which yielded only marginal additional gains, so we retained the current configuration as a good compromise between accuracy and model complexity.

III. FASTTEXT-BASED MODEL 🔗

A. Motivation

Besides GloVe, we also experiment with FastText embeddings² trained directly on the dataset. Unlike GloVe, which builds word vectors from a global word–word co-occurrence matrix, FastText represents each word as a bag

of character n -grams and learns subword embeddings jointly with word embeddings. This allows the model to better handle misspellings, rare words, and the informal morphology that frequently appears in tweets. For instance, many tweets in our dataset contain creative spellings and elongated forms such as “bestest eveerrr!!!” or “dammm”, hashtags like “#yolo” and “#lifecompleted”, as well as informal slang and contractions (e.g. “tmr”, “u got a lot to say”, or the laughter markers “lol”/“lols”). By sharing subword information across such variants, FastText can produce more stable representations than purely word-level models.

B. Experiment

We use the same preprocessing pipeline as in our improved GloVe setup. The embedding dimension is also set to $k = 75$, so that GloVe and FastText can be compared under similar capacity. Once the FastText embeddings are learned, we construct tweet-level representations using the same mean+max pooling scheme as in the GloVe-based model. For classification, we reuse the MLP architecture that performed best with GloVe.

On the test set, the FastText achieves an accuracy of **83.6%**, improving over the best GloVe (82.9%) and confirming the benefit of subword information on noisy Twitter text. A comparison of these results is also shown in Figure 1.

IV. FINE-TUNE TWEETNLP-SENTIMENT 🔗

A. Motivation

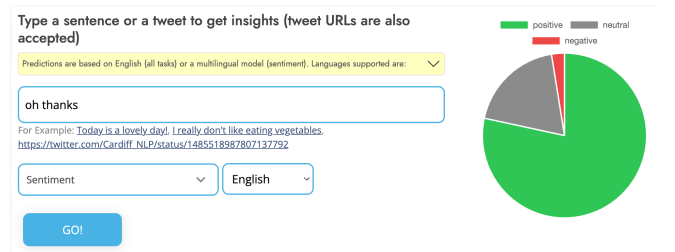


Figure 2. TweetNLP sentiment analysis demo).

Our earlier methods combine static word embeddings with an MLP classifier. Although efficient, they are fundamentally limited for Twitter sentiment: static embeddings cannot adapt word meaning to context, and a shallow classifier struggles to model subtle cues in short, noisy text. These limitations motivate a transition to transformer-based pretrained models, which produce **contextualized token representations** and consistently deliver strong performance across a wide range of NLP tasks. In particular, we leverage the TWEETNLP library³, which provides a suite of transformer models adapted to social-media text and fine-tuned for

²P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, *Enriching Word Vectors with Subword Information*, TACL 2017.

³J. Camacho-Collados et al., *TweetNLP: Cutting-edge Natural Language Processing for Social Media*, 2022; F. Barbieri et al.

multiple tweet-centric tasks (e.g., emotion recognition and emoji prediction). After comparing task alignment and running lightweight preliminary experiments, we select `twitter-roberta-base-sentiment-latest`⁴, a RoBERTa-base model trained on 124M tweets, and finetuned for sentiment analysis with the TweetEval benchmark. This makes it well-suited to Twitter-specific linguistic phenomena such as slang, emojis, and user mentions.

However, the model outputs three labels (*negative*, *neutral*, *positive*; see Figure 2), whereas our task requires binary classification. Therefore an additional mapping from the three-way predictions to our binary label space is needed.

B. Experiments

To adapt the pretrained three-class sentiment model to our binary task, we consider two strategies: **(M1) Logit-based binary mapping**, which retains the original three-class head and derives a binary prediction by comparing the *positive* and *negative* logits; and **(M2) Binary head replacement**, which replaces the original head with a two-class (*positive/negative*) classifier and fine-tunes the model accordingly.

Based on these strategies, we evaluate four experimental settings: (1) **Zero-shot (M1)**, which applies the logit-based mapping without fine-tuning; (2) **Fine-tune (M2)**, which fine-tunes the model with a replaced binary head; (3) **Fine-tune (M1)**, which fine-tunes the full model while retaining the original three-class head and uses logit-based mapping at inference; and (4) **Fine-tune (M1) w/o cleaning**, which mirrors (3) but uses uncleaned inputs with user mentions and hyperlinks restored.

Table I reports the accuracy achieved by each configuration. Zero-shot prediction serves as a baseline, reflecting the model’s out-of-the-box transfer ability. Fine-tuning leads to substantial performance gains across all settings. Among fine-tuned models, retaining the original three-class head with cleaned inputs yields the best performance, suggesting that leveraging the pretrained sentiment structure and reducing input noise both contribute to improved classification.

Table I
ACCURACY OF THE FOUR EXPERIMENTAL SETTINGS WITH
TWEETNLP-SENTIMENT.

Setting	Mapping method	Accuracy
Zero-shot	M1	0.679
Fine-tune	M2	0.903
Fine-tune	M1	0.906
Fine-tune w/o cleaning	M1	0.905

⁴<https://huggingface.co/cardiffnlp/twitter-roberta-base-sentiment-latest>

C. Discussion

Our results suggest that the best performance comes from *preserving* the pretrained sentiment structure and *reducing* input noise. Retaining the original three-class head and applying a binary mapping can be effective because the pretrained classifier has already learned a structured sentiment space, where the *neutral* class acts as a natural intermediate region between *negative* and *positive*. Instead of relearning a binary boundary from scratch, this strategy reuses the existing geometry of the classifier, leading to more stable decisions—especially for weakly polarized tweets. Input cleaning further improves performance by removing tokens that are frequent but largely sentiment-irrelevant, such as user mentions and URLs. These patterns can dominate the token distribution and introduce spurious correlations, which affects not only transformer fine-tuning but also our earlier static-embedding baselines. By reducing such noise, the model can focus more on sentiment-bearing cues and learn from a cleaner training signal.

V. FINE-TUNE BERTWEET-BASE

A. Motivation

While the TweetNLP-sentiment model offers a strong baseline through transfer learning, it introduces a structural task mismatch: it was optimized for ternary classification (*positive*, *neutral*, *negative*) on the TweetEval benchmark, whereas our objective is strictly binary. This necessitates post-hoc mapping strategies (as detailed in Section IV) that may sub-optimally utilize the model’s learned representations.

Therefore, we introduce **BERTweet-base**⁵ to investigate whether *direct fine-tuning of a domain-specific foundation model* yields superior performance. Unlike the TweetNLP checkpoint, BERTweet is pre-trained on a massive corpus of **850 million unlabeled tweets** but has not been specialized for any specific downstream sentiment task. This allows us to:

- **Eliminate Task Misalignment:** We can train a classifier specifically for our binary smiley prediction task from scratch, avoiding the interference of a pre-learned “neutral” class and the need for logit mapping.
- **Leverage Broader Domain Knowledge:** We hypothesize that BERTweet’s significantly larger pre-training corpus (850M tweets compared to standard temporal snapshots) provides a richer understanding of the informal linguistic markers, slang, and emojis prevalent in our dataset.

By fine-tuning this “blank slate” foundation model, we aim to capture the nuances of our specific dataset more effectively than adapting a pre-packaged sentiment classifier.

⁵D. Q. Nguyen, T. Vu, and A. T. Nguyen, *BERTweet: A pre-trained language model for English Tweets*, EMNLP 2020.

B. Experiments

1) *Experimental setup and evaluation protocol:* We evaluate all BERTweet configurations on a held-out **10% validation split**. Detailed hyperparameters are provided in Appendix B, Table VIII. Unless otherwise stated, we monitor validation metrics periodically during training (evaluation every 2000 steps for Base, 4000 for Large), keeping the best checkpoint according to validation accuracy.

Our implementation uses mixed precision (bf16), AdamW, and a warmup ratio of 0.1. We sweep learning rates in $\{1e-5, 2e-5, 3e-5, 5e-5\}$ and find **2e-5** to be the best-performing value for batch size 128 in our runs.

2) *Input normalization aligned with BERTweet pretraining:* To reduce domain shift and make the inputs consistent with the original BERTweet recipe, we apply a lightweight Twitter normalization to match the preprocessing described in the BERTweet resources.

3) *Main validation accuracy: Base, LLRD, and Large:* We experiment with three training strategies, all evaluated on the same 10% validation split:

Table II
VALIDATION ACCURACY FOR MAIN EXPERIMENTS.

Model / Training strategy	Val. accuracy
BERTweet-Base	0.9188
BERTweet-Base + LLRD	0.9186
BERTweet-Large	0.9217

Layer-wise learning rate decay (LLRD). Our LLRD implementation assigns the highest learning rate to the classifier head and applies a geometric decay ($\gamma = 0.95$) towards lower encoder layers (with bias/LayerNorm excluded from weight decay). In our experiments, LLRD does *not* improve validation accuracy over standard fine-tuning at the same base learning rate, suggesting that full-model fine-tuning is already stable and effective for this dataset under our hyperparameter regime.

4) *Overfitting indicators from learning curves:* Across runs, validation accuracy improves early and then saturates, while validation loss decreases up to a point and then increases later in training. This pattern is consistent with overfitting dynamics: continuing optimization reduces training loss but does not translate into improved generalization. For transparency, we provide the smoothed learning curves in Appendix C.

We also test label smoothing to mitigate overfitting. Empirically, we observe that label smoothing can increase validation accuracy even when validation loss increases. This is expected because accuracy depends only on the argmax decision, whereas cross-entropy loss is sensitive to the full predicted probability distribution (i.e., calibration and confidence).

5) *Discussion:* Overall, our BERTweet experiments suggest three robust conclusions under our evaluation protocol: (i) **most gains are obtained within one epoch**

(both Base and Large reach near-peak validation accuracy early); (ii) **increasing model capacity helps modestly**, with BERTweet-Large peaking slightly higher than Base; and (iii) **longer training can hurt** (e.g., the 5-epoch Large run), consistent with the validation-loss reversal observed in the learning curves. Additional techniques such as LLRD and label smoothing are useful scientifically as controlled tests, but under our current settings they do not dominate the performance compared to the core factors: Twitter-specific pretraining, task-aligned binary fine-tuning, and input normalization.

VI. COMPARISON AND CONCLUSION

Table III summarises the test accuracy of all submitted models on the AICrowd leaderboard. The results show a clear progression from static embeddings to pretrained transformers: our improved GloVe and FastText baselines reach **82.9%** and **83.6%** accuracy, while transformer-based fine-tuning yields a substantial jump to above **90%**. In particular, TweetNLP-sentiment reaches **0.906**, and BERTweet provides the best overall performance with **0.920**.

Table III
OVERALL TEST ACCURACY ON THE AICROWD LEADERBOARD.

Model (Config see App. B)	Accuracy
GloVe (improved)	0.829
FastText (improved)	0.836
TweetNLP-sentiment (fine-tuned)	0.906
BERTweet-base (fine-tuned)	0.919
BERTweet-large (fine-tuned)	0.920

Beyond the raw leaderboard results, our experiments suggest two practical takeaways. First, careful preprocessing and normalization consistently improve robustness across model families, from static embeddings to transformers. Second, the transformer gains depend not only on model capacity but also on task alignment: TweetNLP-sentiment is a strong transfer baseline but is trained for ternary sentiment classification, whereas BERTweet is a Twitter-pretrained foundation model that we fine-tune directly for the binary smiley objective. Empirically, BERTweet-Large improves only marginally over BERTweet-Base, indicating diminishing returns in accuracy relative to added compute.

Overall, the project illustrates a clear trade-off. GloVe/FastText models provide simple and efficient baselines that are attractive under strict resource constraints, while pretrained transformers are preferable when maximizing predictive performance on noisy social media text is the primary objective.

APPENDIX

A. Ethical Risks

One ethical risk in our project is systematic misclassification for certain Twitter communication styles. Tweets often include informal markers such as slang, creative spellings, repeated characters, emoji sequences, and platform-specific abbreviations. If these patterns are under-represented in the training data, the model may misread their sentiment strength—e.g., interpreting them as more positive or more negative than intended—which can disproportionately affect users whose writing deviates from the dominant distribution.

To assess this risk concretely, we manually inspected predictions on tweets that exhibit (i) elongated words and repeated characters, (ii) dense emoji usage or emoji combinations, (iii) user-mention- and URL-heavy texts, and (iv) common Twitter abbreviations and laughter markers (e.g., “tmr”, “lol”). We compared these with semantically similar tweets without such markers to identify recurring error patterns and potential spurious cues. Based on observations, we avoided aggressive normalization. Instead, we remove frequent but sentiment-irrelevant tokens such as user mentions and URLs, while preserving sentiment-bearing signals (e.g., emojis and intensifiers) whenever possible.

In addition, we consider a potential privacy-related risk sometimes referred to as a membership inference attack: if a user has a distinctive writing style, an adversary could probe the model with style-matching queries and use its responses to infer whether that user’s data was included in training, which could lead to negative downstream consequences. Possible mitigation strategies include training with differential privacy, limiting or obfuscating confidence scores, and evaluating privacy leakage using dedicated membership-inference benchmarks. However, these techniques require additional privacy-specific tooling, and are therefore beyond the scope of this project.

Finally, because our approach relies on a pretrained model, we acknowledge the possibility of inherited biases from large-scale pretraining data that are not fully transparent. While fine-tuning can improve task performance, it does not guarantee full control over representational biases, so we interpret our findings within the scope of our dataset and task.

B. Model configurations

All the final configurations are determined by cross validation.

1) *GloVe*: The main GloVe embedding hyperparameters are summarised in Table IV, and the MLP hyperparameters are given in Table V..

2) *FastText*: FastText embeddings is trained with the configuration in Table VI. For the classifier, we reuse exactly the same MLP architecture and hyperparameters as in Table V (GloVe-MLP), so that the only difference between the two models lies in the underlying word embeddings.

Table IV
GLOVE EMBEDDING CONFIGURATION.

Parameter	Value	Description
k	75	Word embedding dimension
Initialisation	$N(0, 0.01^2)$	Random normal initialisation
epochs	30	Number of training epochs
batch_size	128	Batch size for GloVe training
η_{base}	0.01	Base learning rate
LR schedule	$\eta_{\text{base}} / (1 + 0.1 \cdot \text{epoch})$	Linear decay schedule
α	0.75	Exponent in weighting function
n_{max}	100	Cut-off for weighting function

Table V
MLP CONFIGURATION FOR THE GLOVE-BASED MODEL.

Parameter	Value	Description
hidden_layer_sizes	(256, 128)	Two hidden layers
activation	"relu"	ReLU activation
max_iter	200	Maximum training iterations
batch_size	256	Mini-batch size
learning_rate	"adaptive"	Learning rate schedule
learning_rate_init	0.001	Initial learning rate
early_stopping	True	Enable early stopping
validation_fraction	0.1	For early stopping
n_iter_no_change	5	Patience for early stopping
alpha	10^{-4}	L2 regularisation strength
random_state	42	Random seed

Table VI
FASTTEXT EMBEDDING CONFIGURATION.

Parameter	Value	Description
model	"skipgram"	Training mode (skip-gram)
dim	75	Embedding dimension k
epoch	20	Number of training epochs
lr	0.05	Learning rate
minn	3	Minimum character n -gram length
maxn	6	Maximum character n -gram length

3) *TweetNLP-sentiment*: The TweetNLP-sentiment is fine tuned with the configuration in Table VII.

Table VII
HYPERPARAMETERS USED DURING FINE-TUNING ON
TWEETNLP-SENTIMENT.

Hyperparameter	Value
Learning rate	2×10^{-5}
Batch size (train / eval)	16 / 32
Gradient accumulation steps	2
Max sequence length	128
Optimizer	AdamW
Weight decay	0.01
Number of epochs	1 (with early stopping)
Early stopping patience	2
Precision	FP16
Checkpointing strategy	Save best model (eval loss)

4) *BERTweet*: We experiment with two variants of BERTweet (base and large) and a specific training strategy involving Layer-wise Learning Rate Decay (LLRD). The

shared hyperparameters for standard fine-tuning are listed in Table VIII.

Table VIII
COMMON HYPERPARAMETERS FOR BERTWEET FINE-TUNING.

Hyperparameter	Value
Tokenizer Normalization	True
Max Sequence Length	128
Batch Size	128
Learning Rate	2×10^{-5}
Optimizer	AdamW
Weight Decay	0.01
Warmup Ratio	0.1
Precision	bf16
Gradient Accumulation	1

For the **LLRD** experiments (based on bertweet-base), we utilize specific decay rates and regularization settings to prevent catastrophic forgetting of the lower layers. These settings are detailed in Table IX.

Table IX
SPECIFIC CONFIGURATION FOR BERTWEET-BASE + LLRD.

Parameter	Value	Description
η_{base}	2×10^{-5}	Base LR (Classifier)
LLRD Decay (γ)	0.95	$\text{lr}^{(l-1)} = \text{lr}^{(l)} \times \gamma$
Label Smoothing	0.1	Regularization
Epochs	5	Training duration

For **BERTweet-Large**, we use the standard hyperparameters from Table VIII but reduce the training duration to 3 epochs due to faster convergence and overfitting risks observed in the learning curves.

Table X
BERTWEET-LARGE VALIDATION ACCURACY ACROSS TRAINING EPOCHS (10% VALIDATION SPLIT).

BERTweet-Large configuration	Val. accuracy
1 epoch	0.920976
3 epochs	0.921676
5 epochs	0.919020

C. BERTweet learning curves

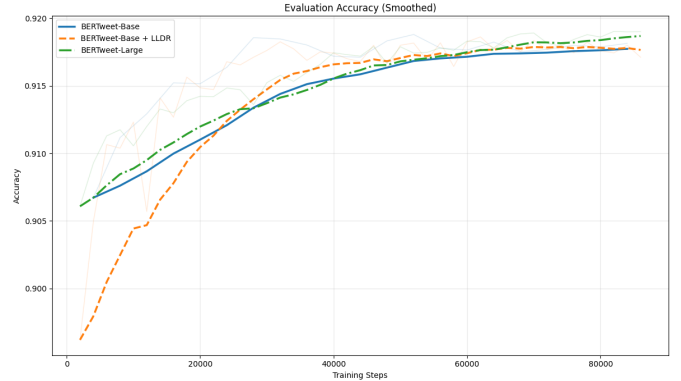


Figure 3. Validation accuracy curves (smoothed) for BERTweet-Base, BERTweet-Base + LLRD, and BERTweet-Large.

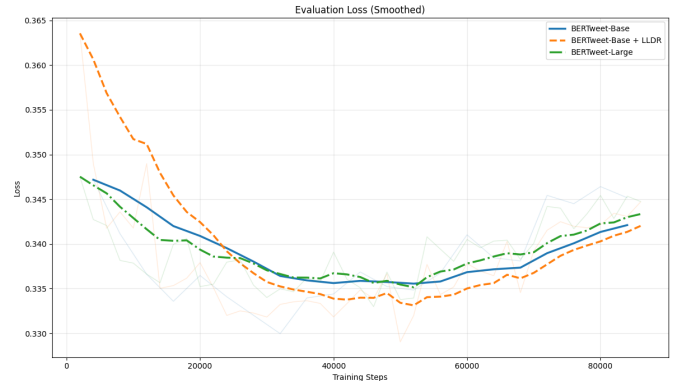


Figure 4. Validation loss curves (smoothed) for BERTweet-Base, BERTweet-Base + LLRD, and BERTweet-Large.